KANT AX/AI 엔지니어 트랙 · 29

Docker 1
컨테이너 실행하기

설치하고, Hub에서 받고,
켜고 끄기

```
~ $ docker run -d --name web -p 8080:80 nginx
ba5cd077165e1aac0f8ca682…

~ $ docker ps
IMAGE   STATUS         PORTS
nginx   Up 1 second    0.0.0.0:8080->80/tcp
```

오늘의 순서

1	**Docker가 뭔가요** 컨테이너, 이미지와 컨테이너, 가상 머신과 비교	3–7쪽
2	**설치하기** Windows WSL 2, Mac 칩 고르기, hello-world	8–14쪽
3	**Docker Desktop과 Docker Hub** 화면 둘러보기, 공식 마크, 태그, docker pull	15–20쪽
4	**컨테이너 켜고 끄기** run, ps, stop, start, restart, rm, -p 8080:80, logs, -it	21–32쪽
5	**자주 나는 오류** 엔진 꺼짐, 켜진 채 rm, 이름, 포트, rmi	33–37쪽
6	**실습** 파이썬 버전 비교, 지우면 사라지는 기록	38–39쪽

40쪽은 한 장 요약이에요.

1. Docker가 뭔가요

kim 컴퓨터에선 되는 코드가 lee 컴퓨터에선 SyntaxError가 나요

main.py

```
menu = {"latte": 4500}
print(f"라떼: {menu["latte"]}원")
```

K kim의 컴퓨터

Python 3.12

```
라떼: 4500원
```

L lee의 컴퓨터

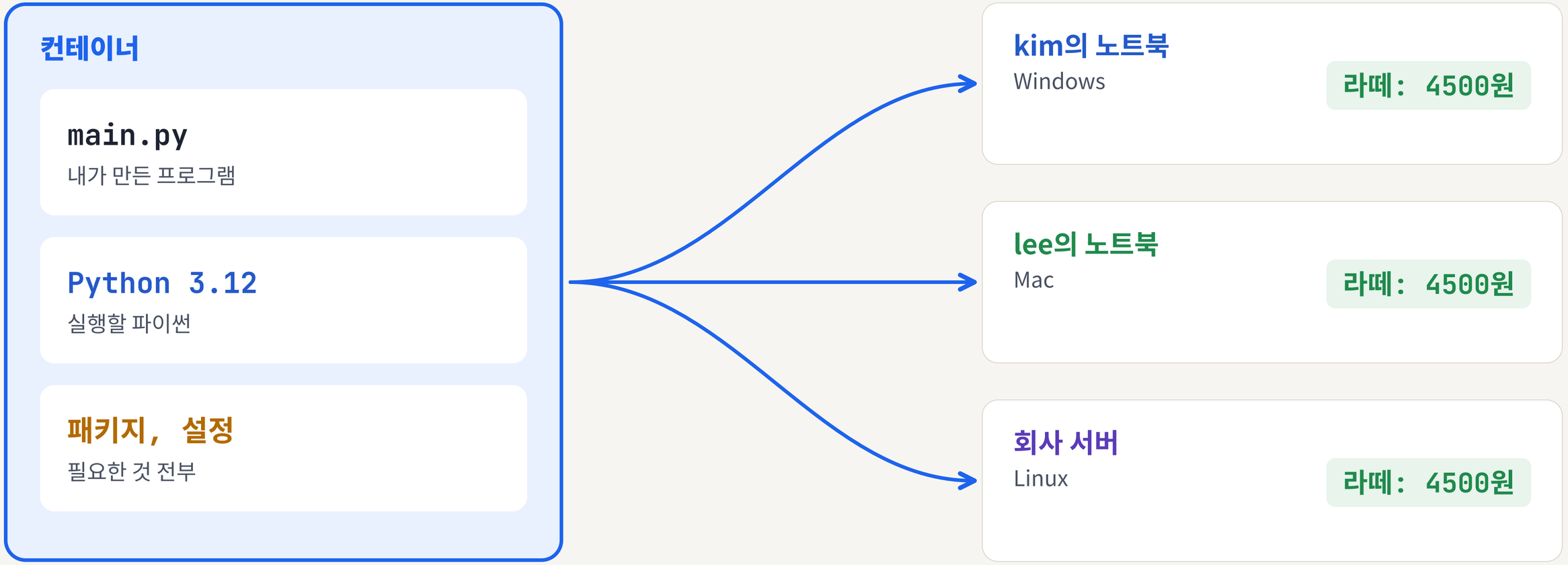
Python 3.11

```
  File "<string>", line 1
    menu = {"latte": 4500}; print(f"라떼: {menu["latte"]}원")
                                                ^^^^^
SyntaxError: f-string: unmatched '['
```

같은 코드인데 컴퓨터마다 설치된 파이썬 버전이 달라요.

1. Docker가 뭔가요

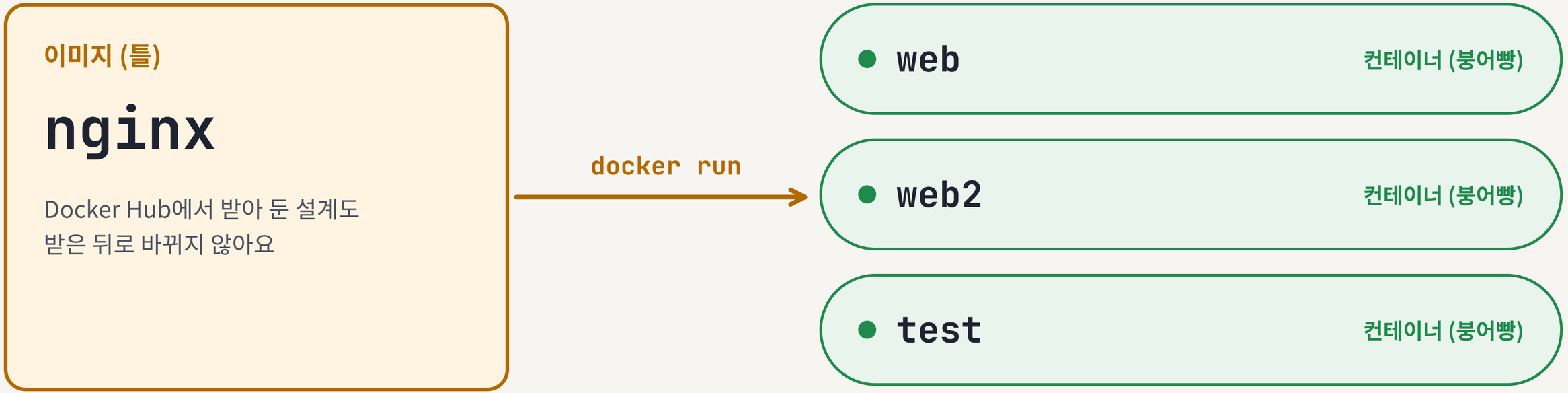
프로그램과 실행 환경을 상자 하나에 담아서 옮겨요



상자째 옮기니까 어디서 켜도 결과가 같아요.

1. Docker가 뭔가요

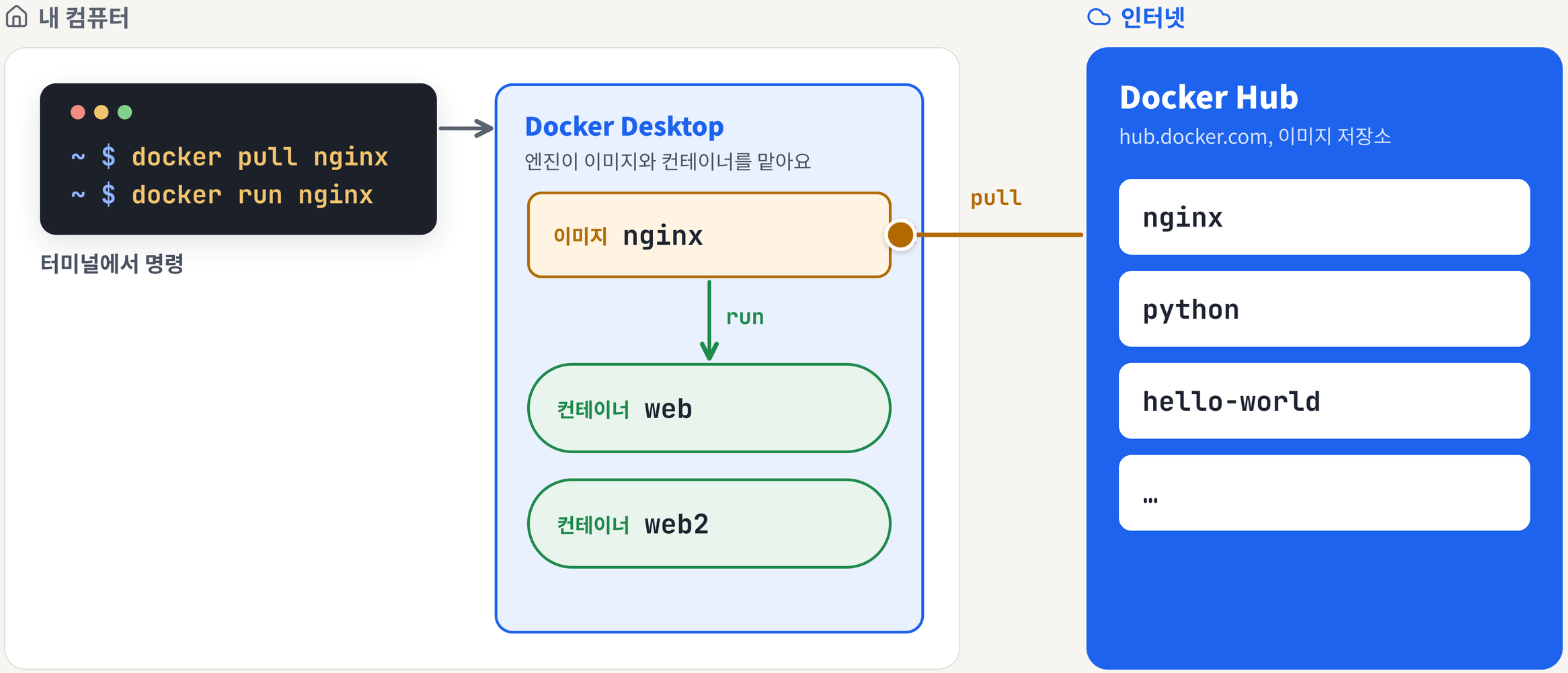
이미지는 붕어빵 틀, 컨테이너는 그 틀로 찍은 붕어빵이에요



이미지	틀, 설계도	`docker images`
컨테이너	틀로 켠 실행 하나	`docker ps`

1. Docker가 뭔가요

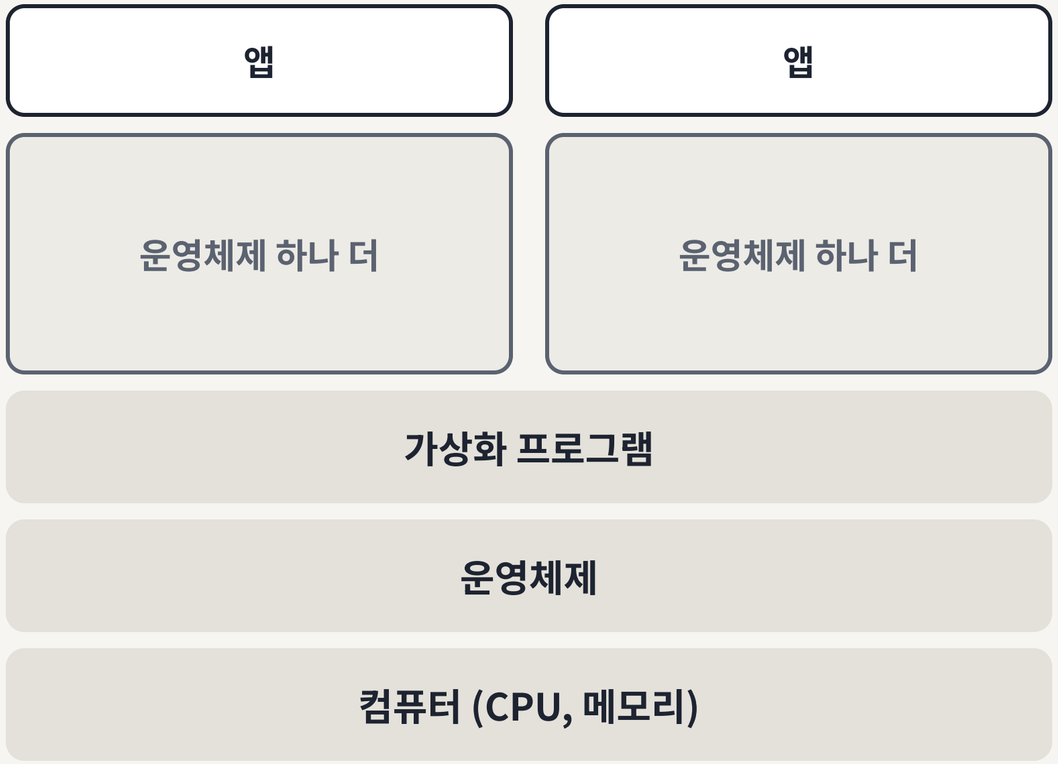
Docker Hub에서 이미지를 받아서 내 컴퓨터에서 컨테이너로 켜요



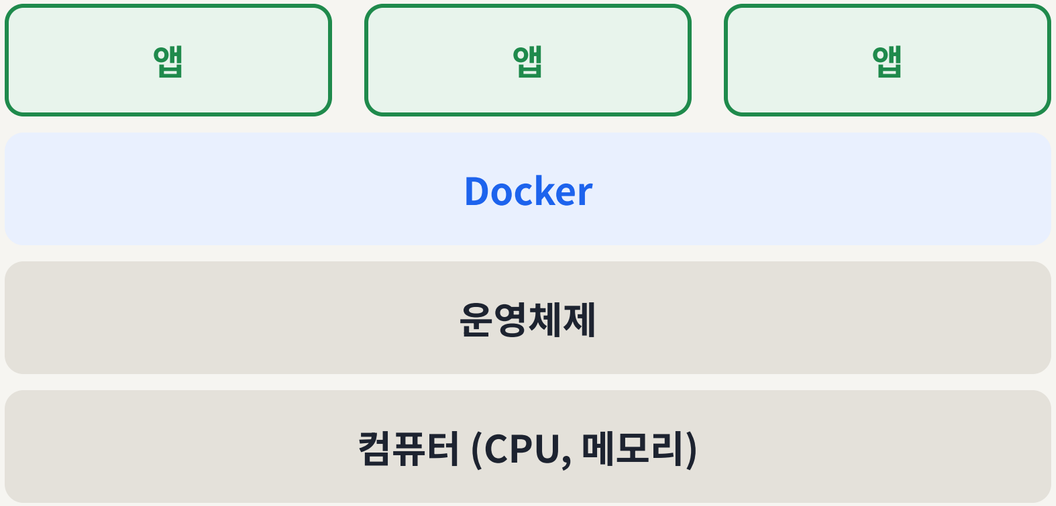
1. Docker가 뭔가요

가상 머신은 컴퓨터를 통째로, 컨테이너는 프로그램만 담아요

가상 머신



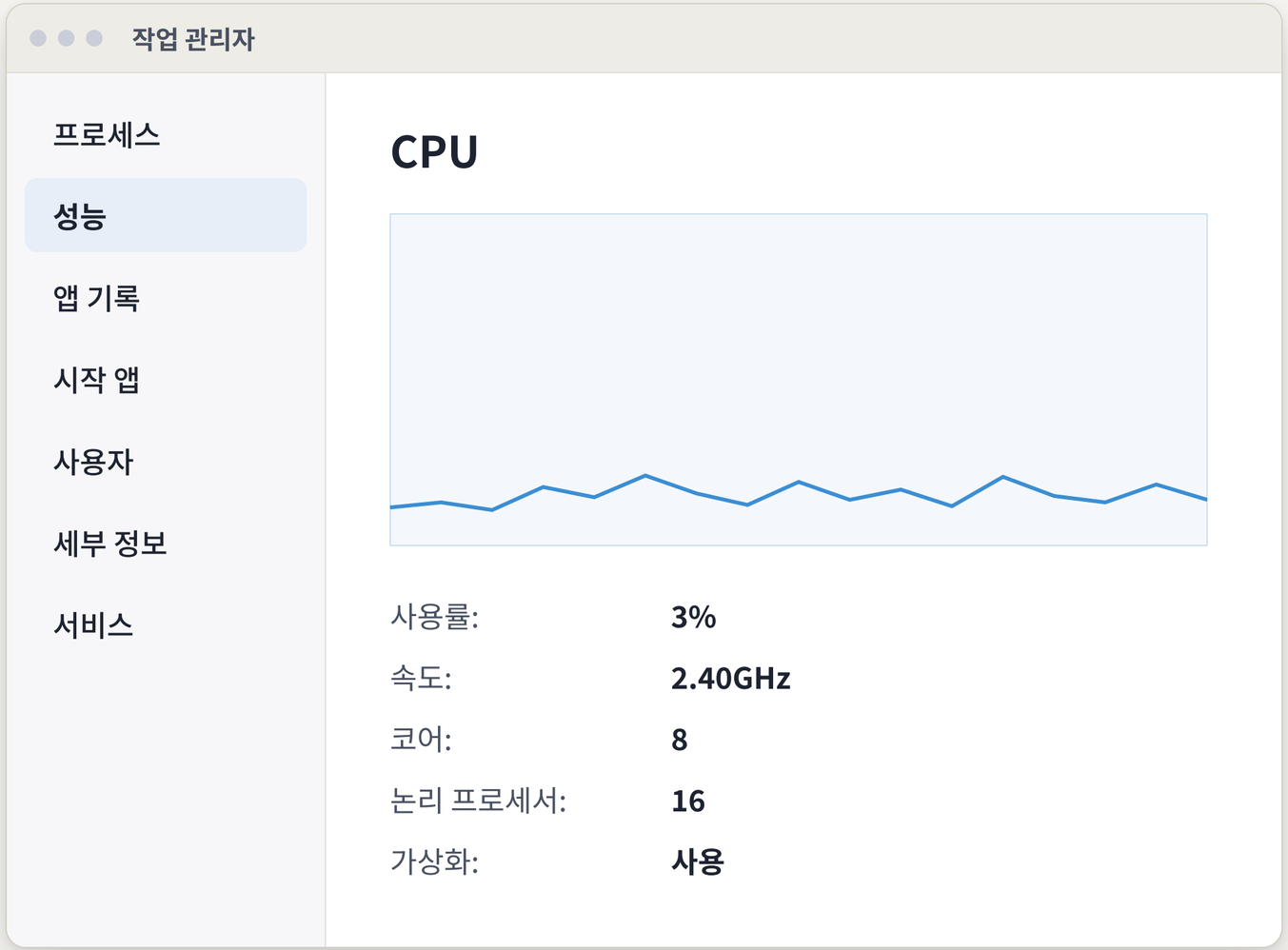
컨테이너



컨테이너는 운영체제를 같이 써서 가볍고 몇 초 만에 켜져요.

2. 설치하기

Windows는 작업 관리자에서 가상화가 켜져 있는지 먼저 봐요



`Ctrl` + `Shift` + `Esc`

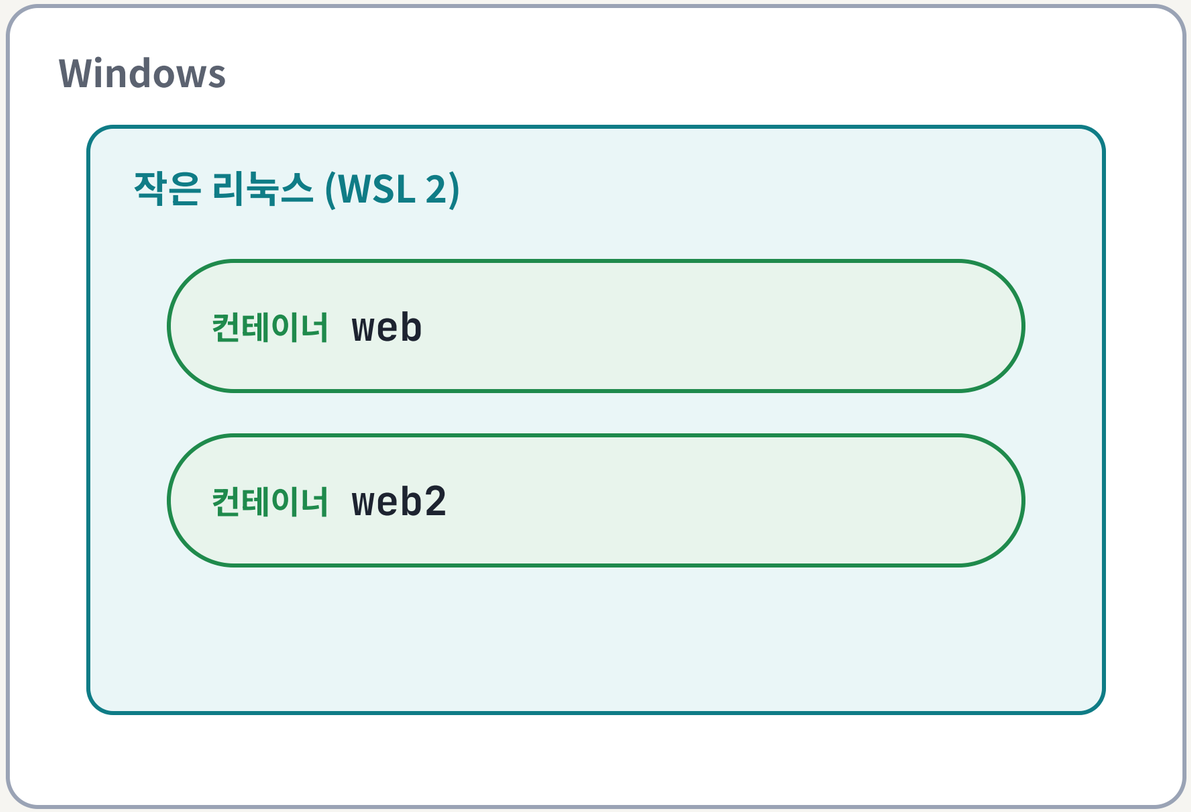
1. Ctrl+Shift+Esc로 작업 관리자 열기
2. 왼쪽 성능, 위의 CPU 누르기
3. 가상화: 사용인지 보기

사용 안 함이면

컴퓨터를 켤 때 BIOS(UEFI) 설정에서 가상화를 켜야 해요.
메뉴가 회사마다 달라서 강사와 같이 해요.

2. 설치하기

WSL 2와 Hyper-V는 Windows 안에 리눅스를 켜는 두 방법이에요



WSL 2 추천

- Windows 안에서 리눅스를 켜는 기능
- Home, Pro 모두 돼요
- Docker Desktop이 기본으로 골라요

Hyper-V

- 가상 컴퓨터를 켜는 Windows 기능
- Pro, Enterprise, Education만 있어요
- WSL 2를 못 쓸 때만 골라요

Docker Desktop 설치 창에 Use WSL 2 instead of Hyper-V 칸이 있으면 켠 채로 둬요.

2. 설치하기

Windows는 WSL을 켠 뒤 Docker Desktop을 설치해요

1

관리자 PowerShell

시작 메뉴에서 PowerShell을 오른쪽 클릭, 관리자 권한으로 실행

```
PS $ wsl --install
```

끝나면 컴퓨터를 다시 시작해요

2

Git Bash 또는 PowerShell

winget으로 Docker Desktop 설치

```
~ $ winget install -e --id Docker.DockerDesktop
```

설치 창이 뜨면 OK, 끝나면 Close

3

다시 시작

컴퓨터를 한 번 더 다시 시작해요

시작 메뉴에서 Docker Desktop을 열어요

2. 설치하기

Mac은 칩 종류를 보고 맞는 설치 파일을 골라요

이 Mac에 관하여

MacBook Air

칩 **Apple M2**

메모리 **16GB**

docker.com에서
Download for Mac - Apple Silicon

이 Mac에 관하여

MacBook Pro

프로세서 **2.4 GHz Intel Core i5**

메모리 **16GB**

docker.com에서
Download for Mac - Intel Chip

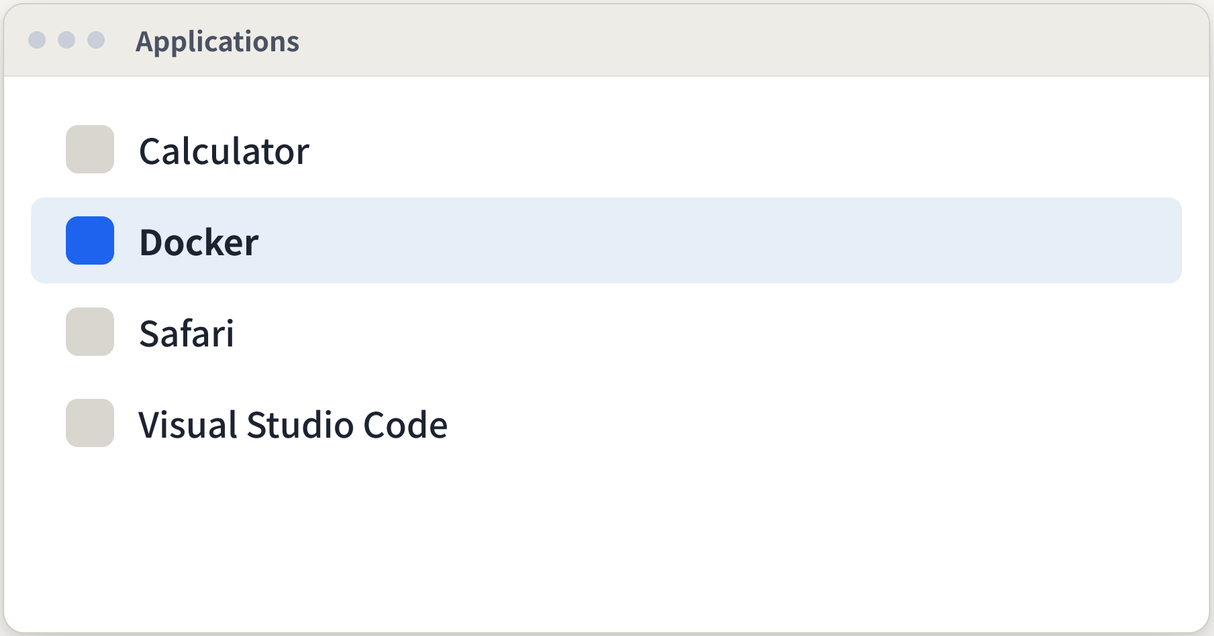
brew로 설치하면 칩에 맞는 파일을 알아서 골라요.

2. 설치하기

Mac은 brew로 설치하고 Applications에서 Docker를 열어요

```
~ $ brew install --cask docker-desktop
```

앱 설치 Docker Desktop

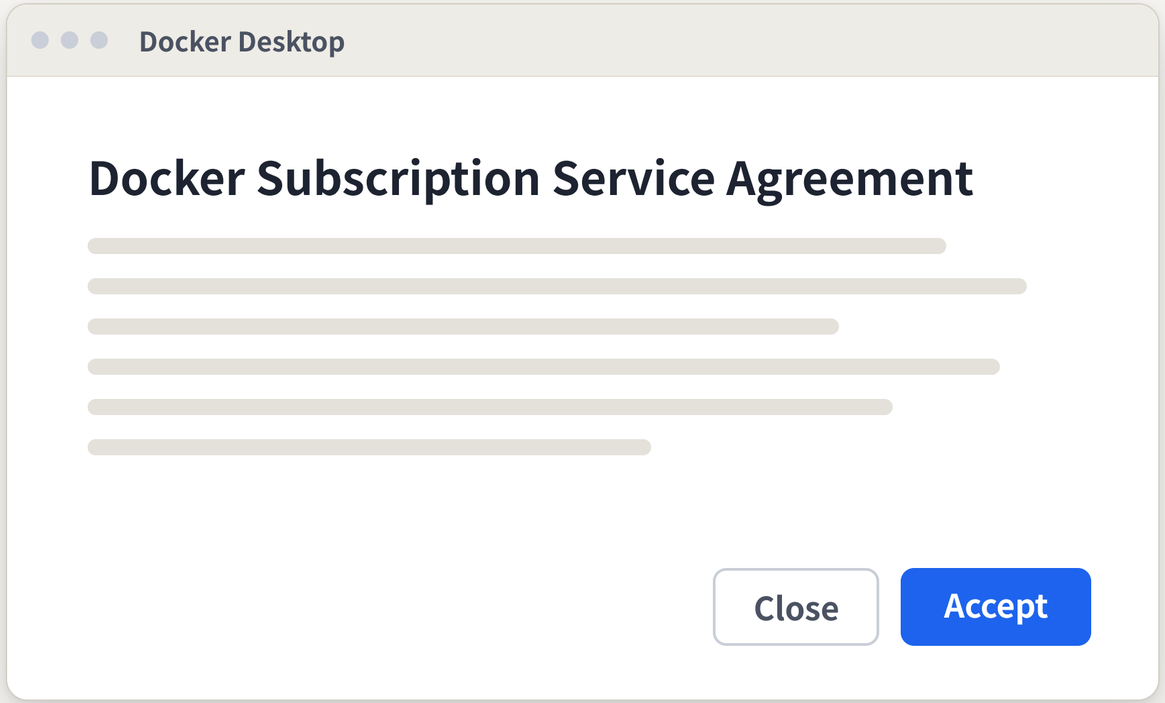


brew 없이 설치할 때

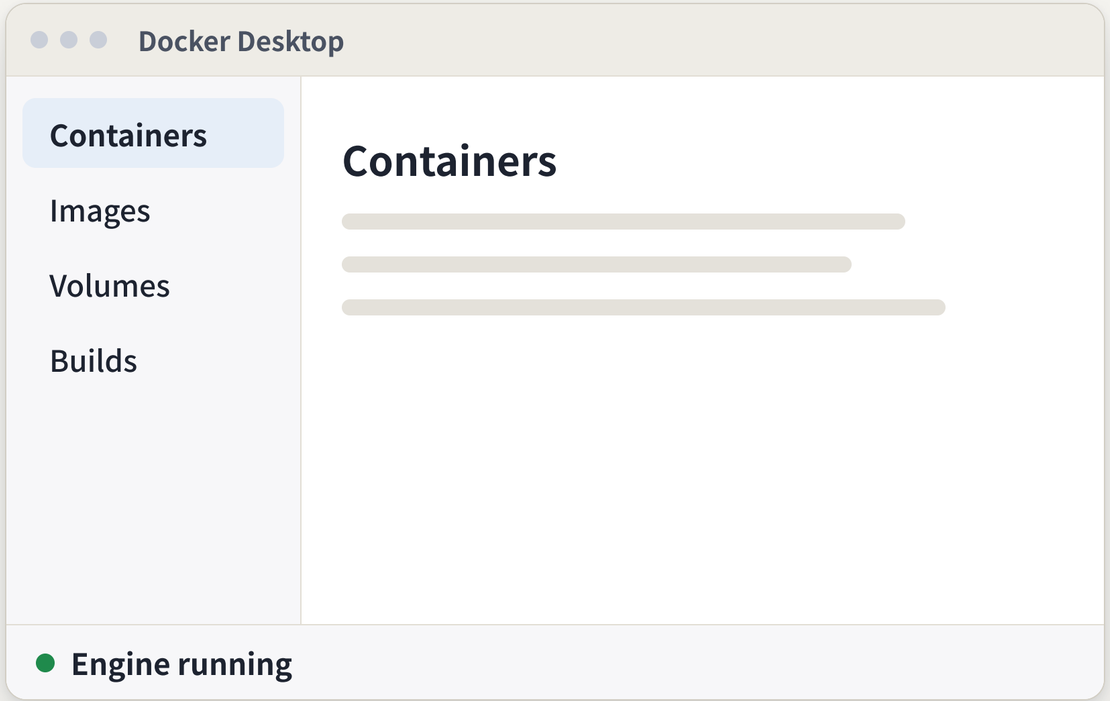
앞 장 버튼으로 Docker.dmg를 받아 열고, Docker 아이콘을 Applications 폴더로 끌어 놓아요.

2. 설치하기

처음 열면 약관에 Accept를 누르고 Engine running을 기다려요



로그인 화면이 나오면 건너뛰어도 돼요.
오늘 받는 이미지는 로그인 없이 받아요.



2. 설치하기

터미널에서 docker --version과 hello-world로 확인해요

```
~ $ docker --version
Docker version 29.9.2, build a82e528
~ $ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
Digest: sha256:c567b231f65614688711a75f31518a064258a51d442117a3e771a3a8fdba78ef
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
…
```

설치된 버전

이미지가 없어서 Hub에서 받아요

설치 성공

Windows는 Git Bash, Mac은 터미널에서 쳐요. 버전 숫자는 달라도 괜찮아요.

3. Docker Desktop과 Docker Hub

Docker Desktop은 왼쪽 메뉴 두 개만 먼저 기억해요

컨테이너 목록

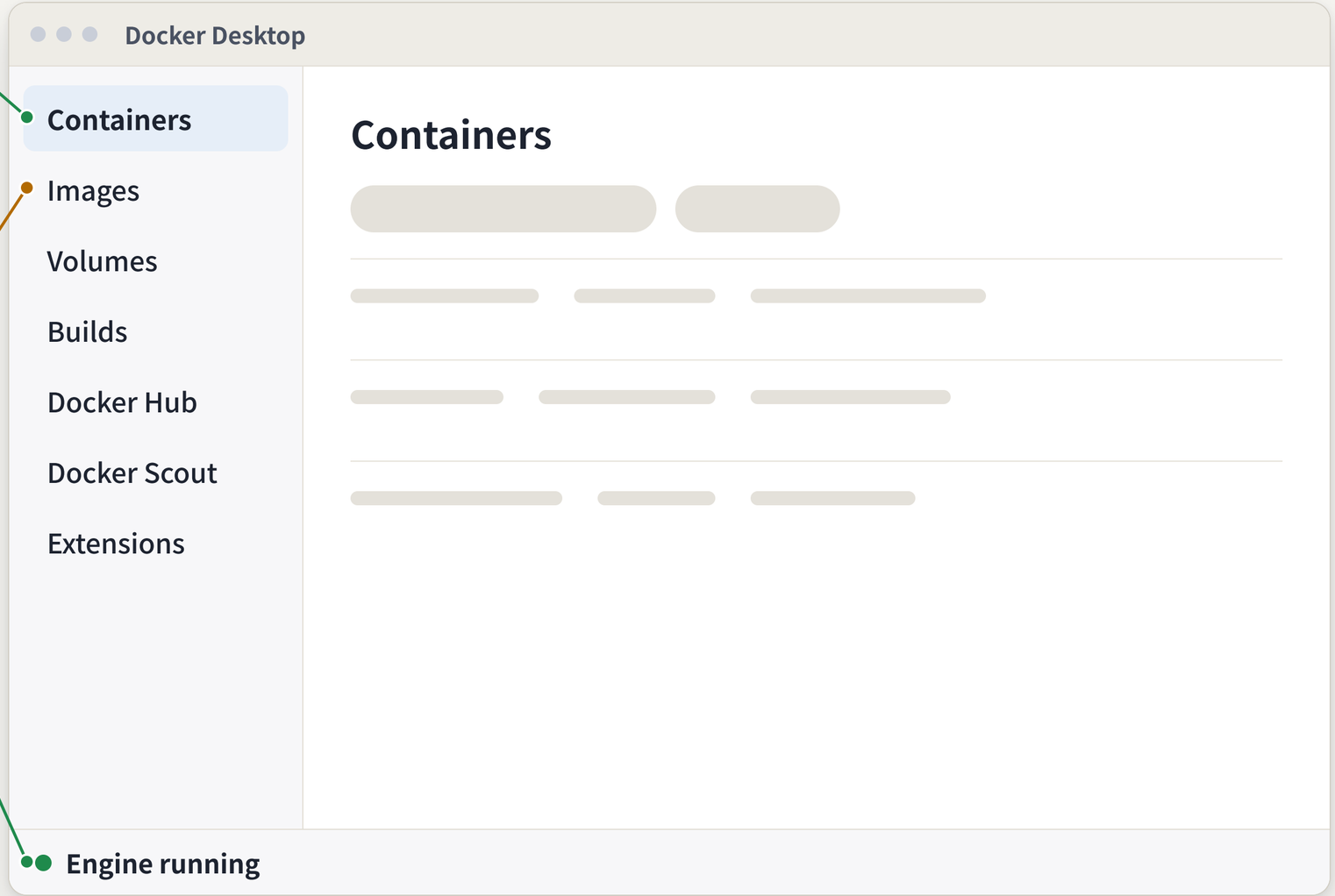
터미널: `docker ps -a`

받은 이미지 목록

터미널: `docker images`

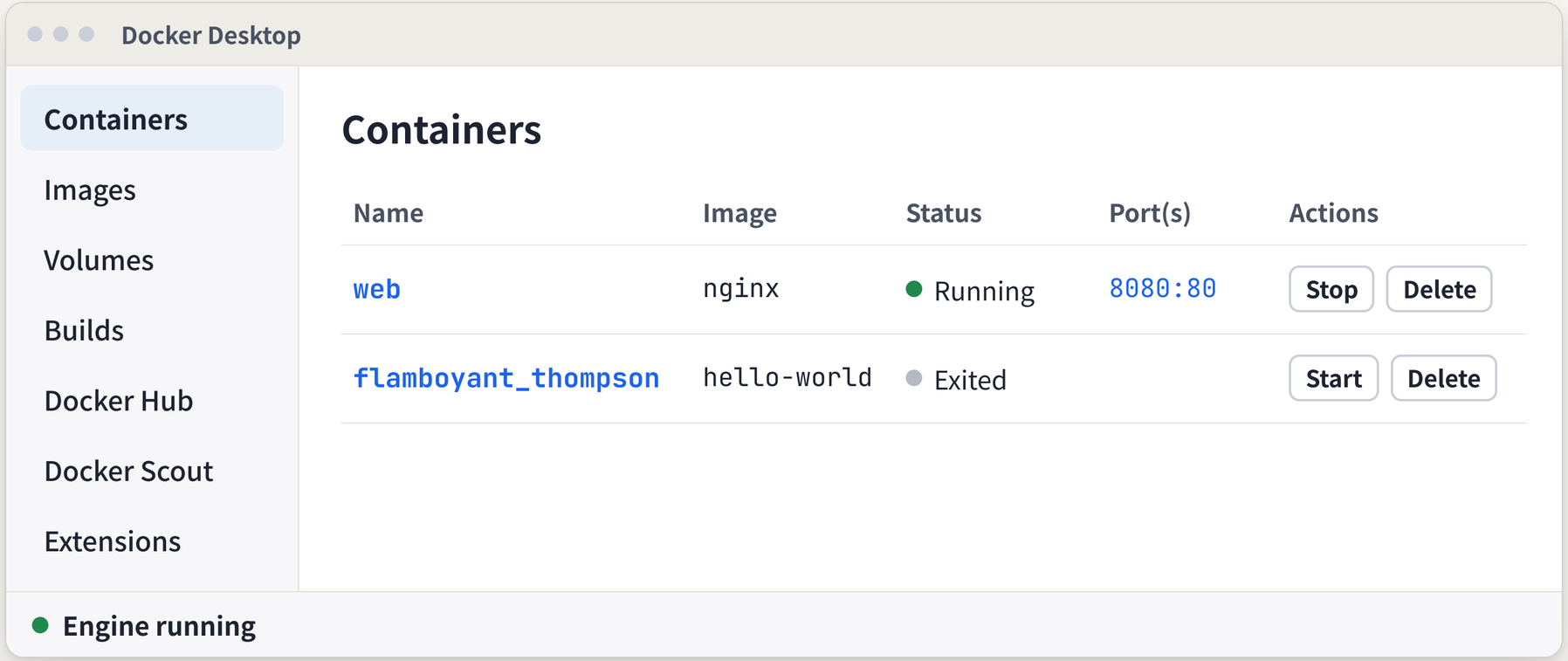
엔진 상태

Engine running이어야 명령이 돼요



3. Docker Desktop과 Docker Hub

Containers 화면으로 상태를 보고, 명령은 터미널에서 쳐요



Stop `docker stop web`

Start `docker start 이름`

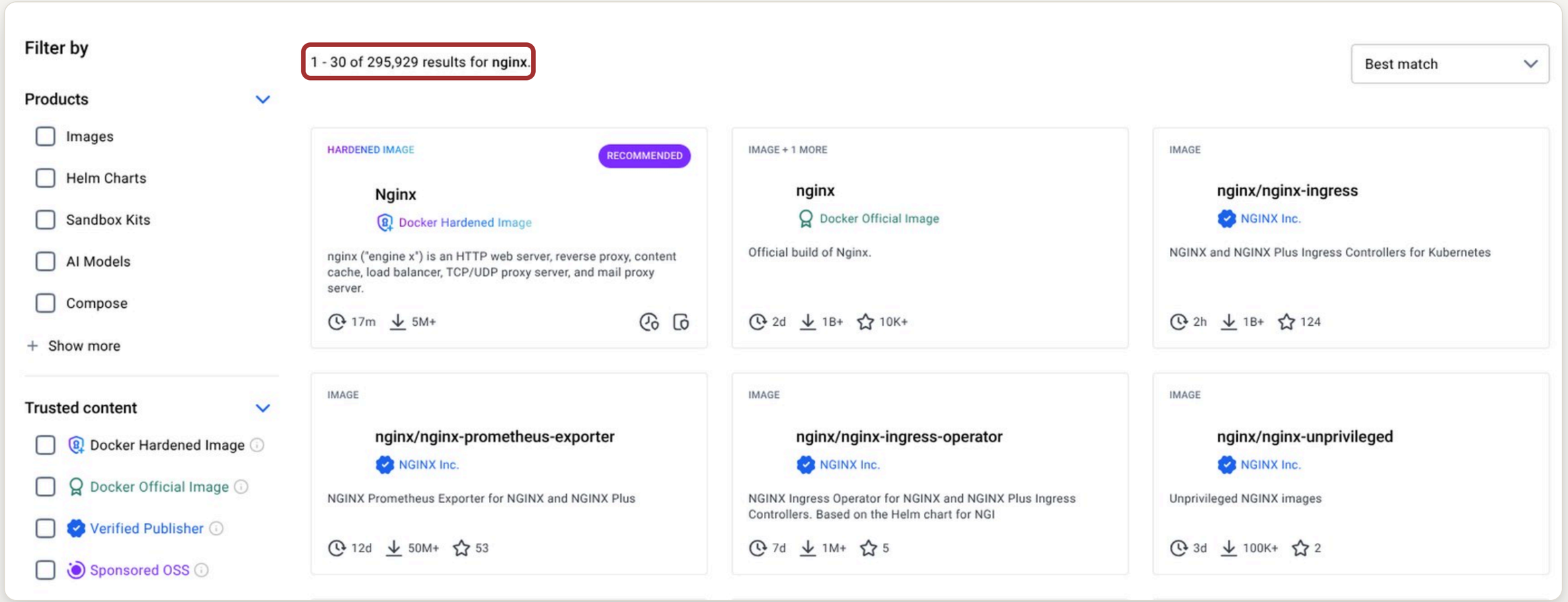
Delete `docker rm 이름`

버튼과 명령이 하는 일이 같아요.

수업에서는 터미널 명령으로 하고, 결과는 이 화면으로 확인해요.

3. Docker Desktop과 Docker Hub

hub.docker.com에서 검색하면 같은 이름이 수십만 개 나와요



이름만 보고 고르지 않고, 이름 아래 마크를 봐요.

3. Docker Desktop과 Docker Hub

Docker Official Image 마크가 붙은 이미지부터 골라요

Docker Official Image	Docker가 직접 관리하는 기본 이미지 `nginx, python, ubuntu`
Verified Publisher	회사가 직접 올린 이미지 `nginx/nginx-ingress (NGINX Inc.)`
Sponsored OSS	Docker가 후원하는 오픈소스 프로젝트
Docker Hardened Image	보안을 더 다듬은 이미지
⚠ 마크 없음	누구나 올린 이미지, 안에 무엇이 있는지 몰라요

`nginx` 공식 이미지는 이름에 / 가 없어요. **회사/이름** 다른 이미지는 앞에 올린 곳이 붙어요.

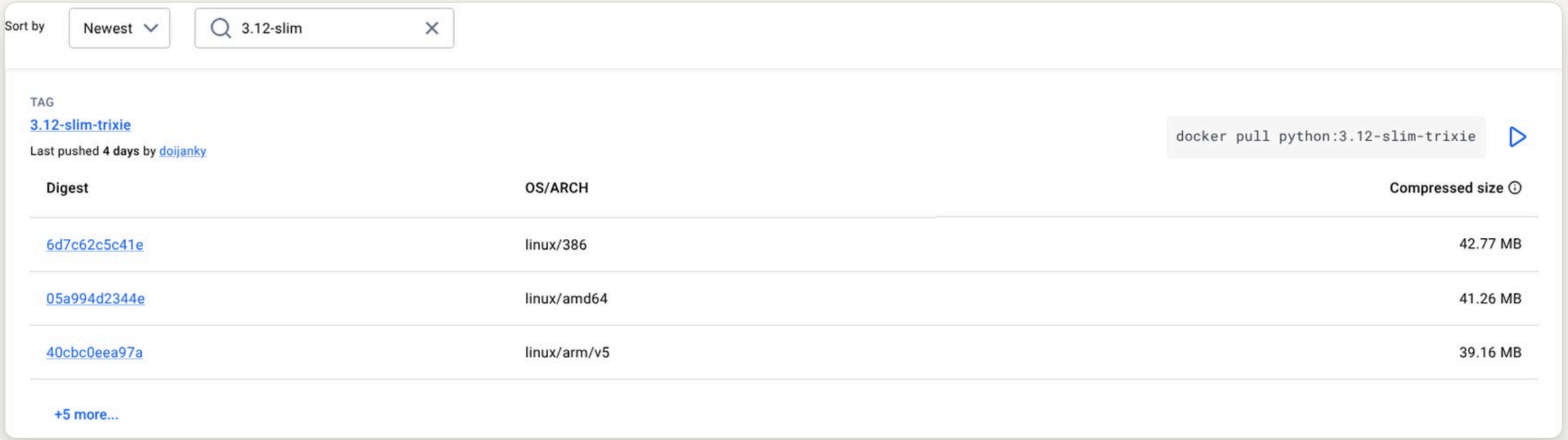
3. Docker Desktop과 Docker Hub

이미지 이름 뒤 콜론 다음이 태그예요. 태그로 버전을 골라요

python:3.12-slim

이미지 이름 | 태그 (버전)

nginx	태그를 안 쓰면 latest (최신)
python:3.12	파이썬 3.12
python:3.12-slim	필요 없는 것을 뺀 작은 버전



docker pull로 받고 docker images로 받은 이미지를 봐요

```
~ $ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
6d2e43921e9e Pulling fs layer
…
Digest: sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

~ $ docker images
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
hello-world:latest   c567b231f656       25.9kB         9.49kB   U
nginx:latest         165ddde8751e        243MB         66.8MB
```

태그를 안 써서 latest

이미지는 여러 층으로 받아요

받아 둔 이미지

4. 컨테이너 켜고 끄기

docker run nginx는 터미널을 붙잡고 끝나지 않아요

```
~ $ docker run nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
…
2026/11/12 00:33:29 [notice] 1#1: nginx/1.31.8
2026/11/12 00:33:29 [notice] 1#1: start worker processes
^C   (Ctrl+C)
2026/11/12 00:33:32 [notice] 1#1: signal 2 (SIGINT) received, exiting
…
```

서버라서 계속 켜져 있어요

Ctrl+C를 받고 꺼져요

hello-world는 글을 찍고 바로 끝났지만,
nginx는 웹 서버라서 멈출 때까지 켜져 있어요.

Ctrl + C

4. 컨테이너 켜고 끄기

docker ps -a에는 꺼진 컨테이너도 남아 있어요

```
~ $ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

~ $ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS                          PORTS     NAMES
a669a15ddb87   nginx         "/docker-entrypoint.…"   3 seconds ago   Exited (0) Less than a second ago             exciting_mendeleev
ae0d403e2768   hello-world   "/hello"                 6 seconds ago   Exited (0) 6 seconds ago                      flamboyant_thompson
```

켜진 컨테이너만: 없어요

이름을 안 주면 Docker가 지어 줘요

exciting_mendeleev처럼 형용사와 과학자 이름을 섞어요. 다음 장에서 이름을 직접 붙여요.

Exited는 꺼졌다는 뜻

꺼져도 지우기 전까지 목록에 남아요. Docker Desktop Containers 화면도 같아요.

4. 컨테이너 켜고 끄기

-d로 뒤에서 켜고, --name으로 이름을 붙여요

```
docker run -d --name web nginx
           |        |        |
           |        |        이미지
           |        이름 붙이기
           뒤에서 켜기
```

```
~ $ docker run -d --name web nginx
b4a5370848e46a4d81e6fee1eab39198bc96338563594339734 7efcf4d87cd13
~ $ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED                  STATUS                  PORTS     NAMES
b4a5370848e4   nginx     "/docker-entrypoint.…"   Less than a second ago   Up Less than a second   80/tcp    web
```

컨테이너 ID, 바로 프롬프트가 돌아와요

Up은 켜져 있다는 뜻이에요. 이제 이름 web으로 부를 수 있어요.

4. 컨테이너 켜고 끄기

docker stop으로 끄고 docker start로 다시 켜요

```
~ $ docker stop web
web
~ $ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
~ $ docker ps -a
CONTAINER ID   IMAGE         COMMAND                  CREATED         STATUS                          PORTS     NAMES
b4a5370848e4   nginx         "/docker-entrypoint.…"   1 second ago    Exited (0) Less than a second ago             web
a669a15ddb87   nginx         "/docker-entrypoint.…"   4 seconds ago   Exited (0) Less than a second ago             exciting_mendeleev
ae0d403e2768   hello-world   "/hello"                 7 seconds ago   Exited (0) 6 seconds ago                      flamboyant_thompson
~ $ docker start web
web
~ $ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS                  PORTS     NAMES
b4a5370848e4   nginx     "/docker-entrypoint.…"   1 second ago    Up Less than a second   80/tcp    web
```

켜진 것이 없어요

꺼졌어요

다시 켜졌어요

stop은 끄기만 해요. 컨테이너는 목록에 남아서 start로 다시 켤 수 있어요.

4. 컨테이너 켜고 끄기

docker restart는 끄고 켜기를 한 번에 해요

`docker restart web` = `docker stop web` + `docker start web`

```
~ $ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS     NAMES
315ddf61831a   nginx     "/docker-entrypoint.…"   20 seconds ago   Up 20 seconds   80/tcp    web
~ $ docker restart web
web
~ $ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS                  PORTS     NAMES
315ddf61831a   nginx     "/docker-entrypoint.…"   21 seconds ago   Up Less than a second   80/tcp    web
```

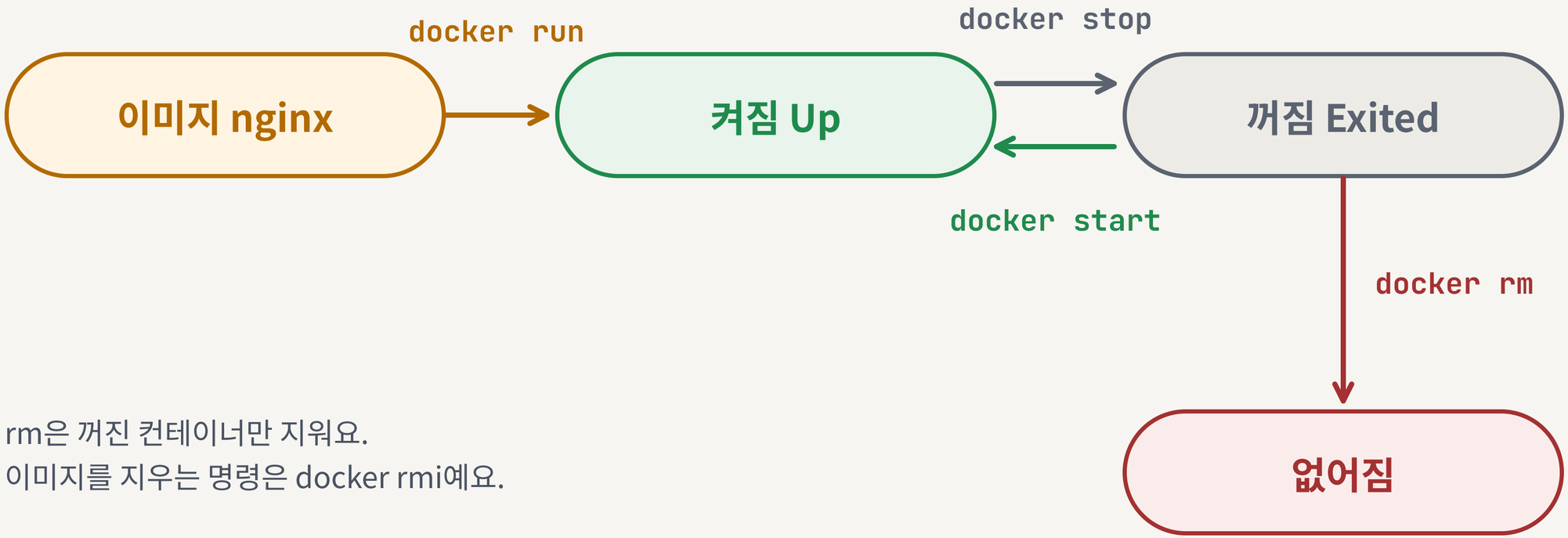
끄고 다시 켰어요

STATUS는 처음부터 다시 세고, CREATED는 그대로예요. 새 컨테이너가 아니라 같은 컨테이너를 다시 켠 거예요.

4. 컨테이너 켜고 끄기

컨테이너는 켜짐과 꺼짐을 오가다가 rm으로 사라져요

run은 새 컨테이너를 만들어서 켜요.
start는 꺼 둔 컨테이너를 다시 켜요.



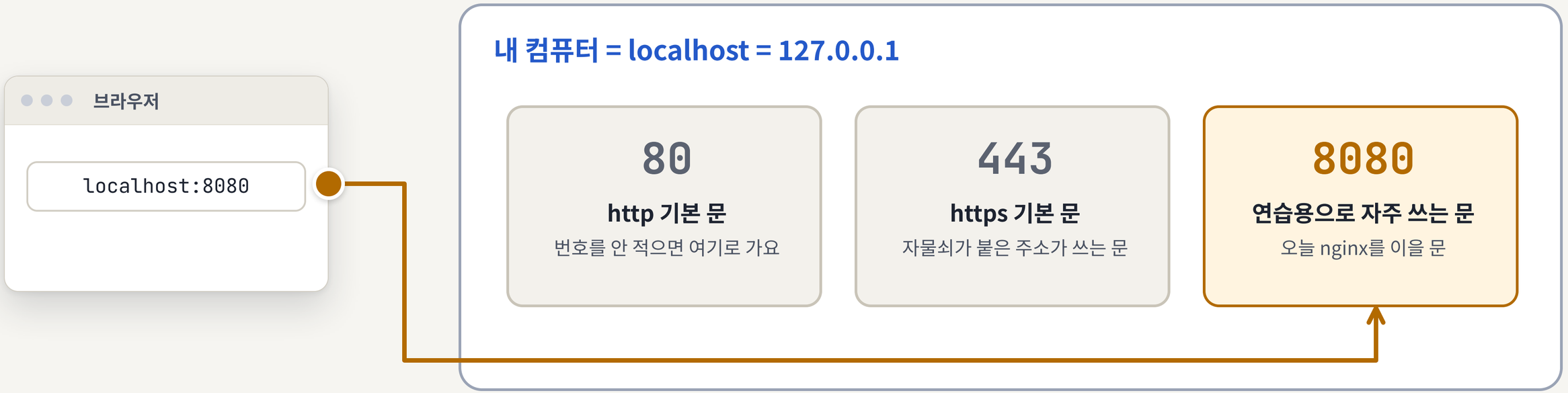
rm은 꺼진 컨테이너만 지워요.
이미지를 지우는 명령은 docker rmi예요.

4. 컨테이너 켜고 끄기

localhost는 내 컴퓨터, 8080은 그 안의 문 번호예요

```
http://localhost:8080
```

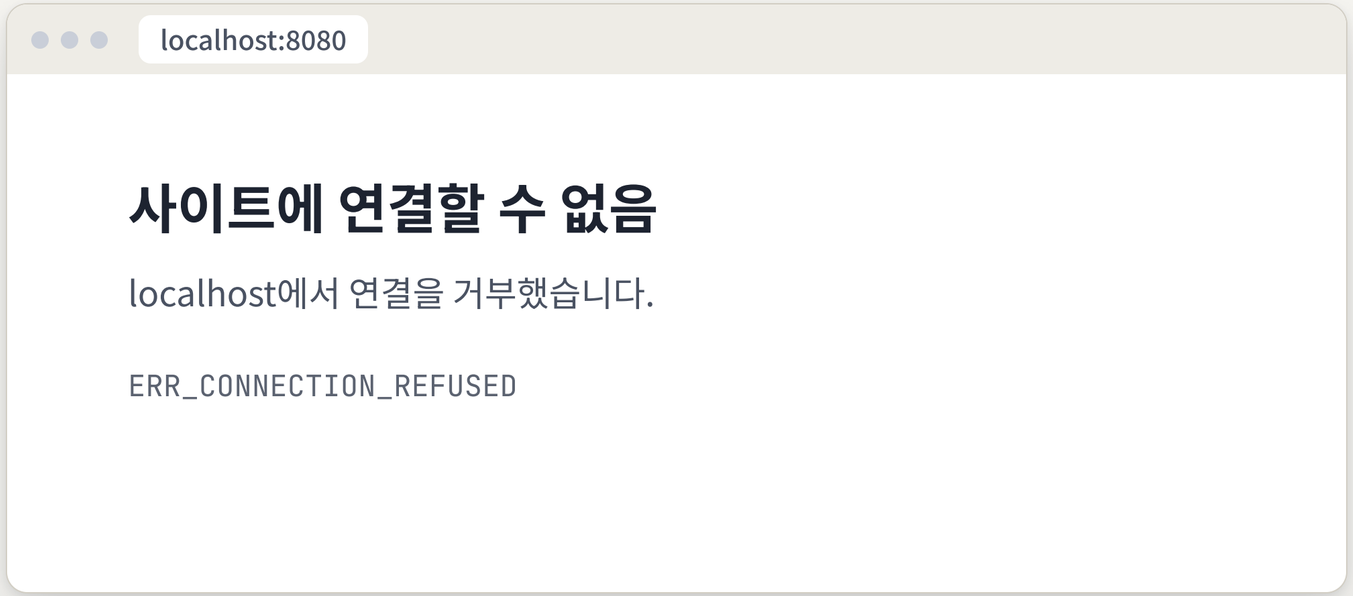
- `http://` — 웹 주소 약속
- `localhost` — 내 컴퓨터
- `:8080` — 문 번호 (포트)

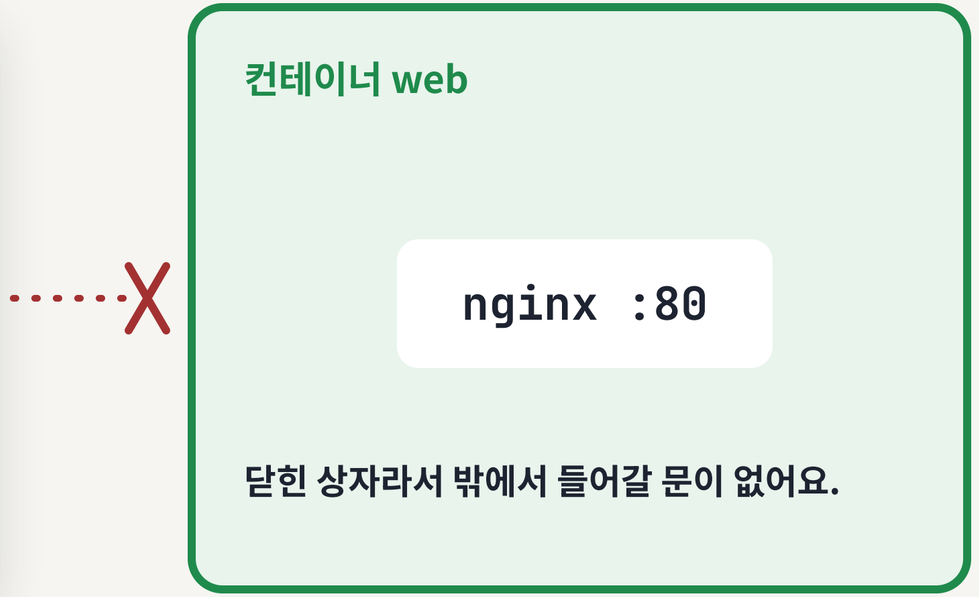


한 컴퓨터에서 여러 프로그램이 요청을 기다려서, 문 번호로 어느 프로그램에 갈지 정해요.

4. 컨테이너 켜고 끄기

켜져 있는 nginx에 브라우저로 들어가면 안 열려요





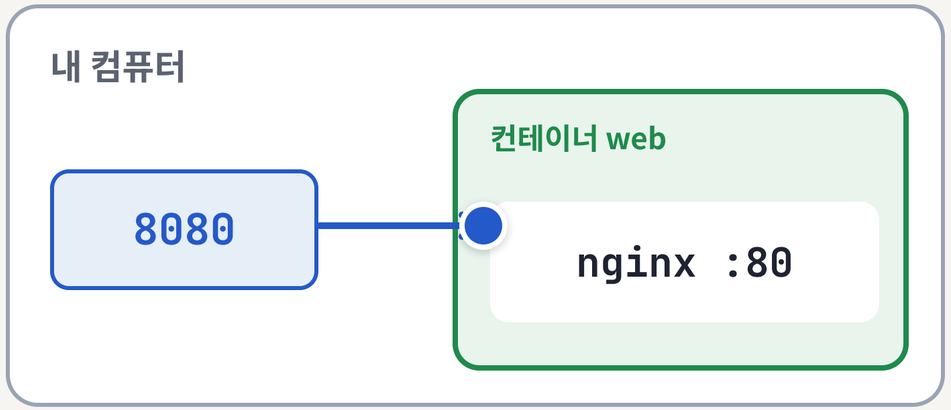

```
~ $ curl http://localhost:8080
curl: (7) Failed to connect to localhost port 8080 after 0 ms: Couldn't connect to server
```

4. 컨테이너 켜고 끄기

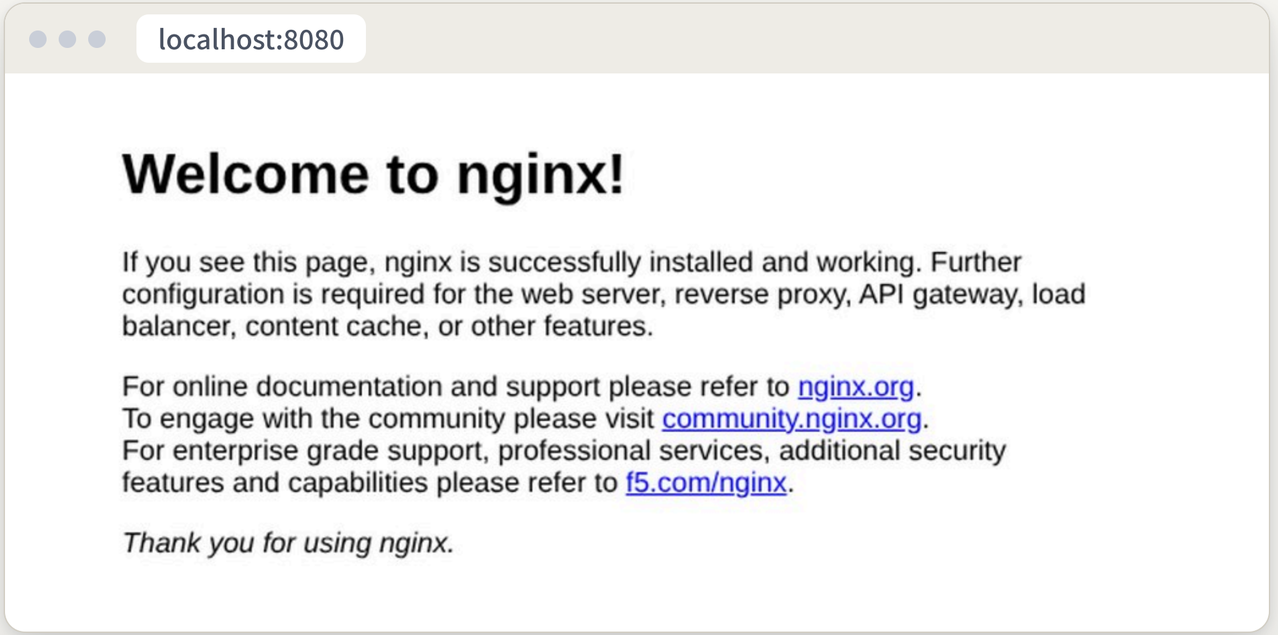
-p 8080:80은 내 컴퓨터 8080번 문을 컨테이너 80번 문에 이어요

```
docker run -d --name web -p 8080:80 nginx
```

내 컴퓨터 문 | 컨테이너 문



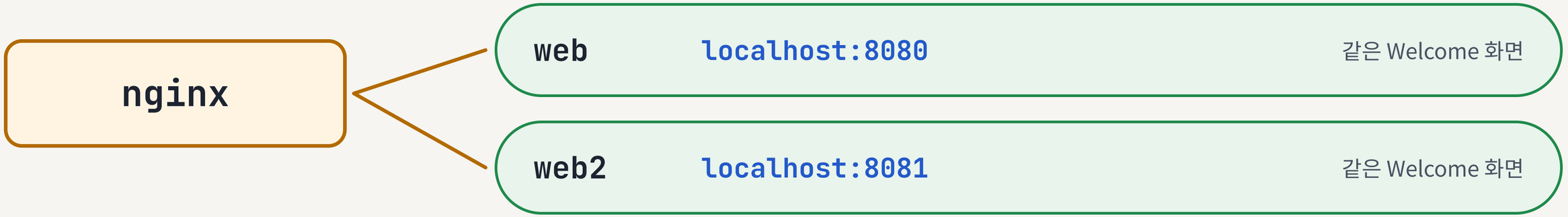

```
PORTS                    NAMES
0.0.0.0:8080->80/tcp     web
```



4. 컨테이너 켜고 끄기

같은 이미지로 두 개를 켤 때는 이름과 내 컴퓨터 문을 다르게 줘요

```
~ $ docker run -d --name web2 -p 8081:80 nginx
ddfc59cdbbe5515a3982b4c87392750a464955f6942d5960a48bd9fa2ff20c08
~ $ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED                  STATUS                  PORTS                    NAMES
ddfc59cdbbe5   nginx     "/docker-entrypoint.…"   Less than a second ago   Up Less than a second   0.0.0.0:8081->80/tcp     web2
ba5cd077165e   nginx     "/docker-entrypoint.…"   2 seconds ago            Up 1 second             0.0.0.0:8080->80/tcp     web
```



4. 컨테이너 켜고 끄기

docker logs로 컨테이너가 남긴 기록을 봐요

```
~ $ docker logs web
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
…
2026/11/12 00:33:33 [notice] 1#1: start worker process 20
172.17.0.1 - - [12/Nov/2026:00:33:34 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.5.0" "-"
172.17.0.1 - - [12/Nov/2026:00:33:35 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.5.0" "-"
172.17.0.1 - - [12/Nov/2026:00:33:35 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.5.0" "-"
172.17.0.1 - - [12/Nov/2026:00:33:35 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.5.0" "-"
```

localhost:8080에 4번 들어온 기록이에요.

GET /는 첫 화면을 달라는 요청, 200은 잘 보냈다는 뜻이에요. Docker Desktop에서 컨테이너 이름을 누르면 같은 기록이 보여요.

4. 컨테이너 켜고 끄기

-it로 우분투 컨테이너 안에 들어가서 명령을 쳐요

```
docker run -it --name my-server ubuntu:24.04 bash
```

- `-it` : 키보드로 치고 화면으로 보기
- `--name my-server` : 이름 붙이기
- `ubuntu:24.04` : 이미지:태그
- `bash` : 안에서 켤 프로그램

```
~ $ docker run -it --name my-server ubuntu:24.04 bash
Unable to find image 'ubuntu:24.04' locally
…
root@01040a1eda9b:/# cat /etc/os-release | head -2
PRETTY_NAME="Ubuntu 24.04.5 LTS"
NAME="Ubuntu"
root@01040a1eda9b:/# whoami
root
root@01040a1eda9b:/# exit
exit
```

root@로 바뀌면 컨테이너 안이에요

exit로 나오면 컨테이너도 꺼져요

내 컴퓨터가 Windows나 Mac이어도 01에서 배운 리눅스 명령을 바로 쳐 볼 수 있어요.

5. 자주 나는 오류

failed to connect to the docker API가 나오면 Docker Desktop을 켜요

```
~ $ docker ps
failed to connect to the docker API at unix:///Users/kim/.docker/run/docker.sock; check if the path
is correct and if the daemon is running: dial unix /Users/kim/.docker/run/docker.sock: connect: no
such file or directory
```

뜻 Docker Desktop(엔진)이 꺼져 있어서 명령을 받을 곳이 없어요.

고치기

```
Docker Desktop 열기
왼쪽 아래 Engine running 기다리기
~ $ docker ps
```

컴퓨터를 다시 켠 뒤에 자주 나요. 엔진이 켜지는 동안 1~2분 기다려요.

5. 자주 나는 오류

켜진 컨테이너는 rm으로 바로 못 지워요

```
~ $ docker rm web
Error response from daemon: cannot remove container "web": container is running: stop the container
before removing or force remove
```

뜻 **web이 아직 켜져 있어요. 끄고 나서 지워요.**

고치기

```
~ $ docker stop web
web
~ $ docker rm web
web
```

지운 컨테이너는 되살릴 수 없어요. 이미지는 남아 있어서 docker run으로 새로 켜면 돼요.

5. 자주 나는 오류

Conflict. The container name이 나오면 이름이 겹친 거예요

```
~ $ docker run -d --name web -p 8080:80 nginx
docker: Error response from daemon: Conflict. The container name "/web" is already in use by
container "ba5cd077165e1aac0f8ca6822ed77da447719a626a1aca498dbb8fa2cec90ae5". You have to remove (or
rename) that container to be able to reuse that name.

Run 'docker run --help' for more information
```

뜻 **web이라는 컨테이너가 이미 있어요. 꺼져 있어도 이름은 그대로 차지해요.**

고치기

```
~ $ docker ps -a
web이 있는지 보고
~ $ docker stop web
~ $ docker rm web
```

지우기 싫으면 --name web3처럼 다른 이름으로 켜요.

5. 자주 나는 오류

port is already allocated가 나오면 내 컴퓨터 문이 이미 쓰이고 있어요

```
~ $ docker run -d --name web2 -p 8080:80 nginx
f637c2dda017f745292be9af113c3e567b511e214ff9de54e4fb5e2bfae5515c
docker: Error response from daemon: failed to set up container networking: driver failed programming
external connectivity on endpoint web2
(60b6f6d0dcc37d67c621705ad7a0fcfbcf22df285eee9b407bbc13b7cbab6784): Bind for 0.0.0.0:8080 failed:
port is already allocated

Run 'docker run --help' for more information
```

뜻 8080번 문은 web이 쓰고 있어요. 한 문에는 하나만 이을 수 있어요.

고치기

```
~ $ docker rm web2
~ $ docker run -d --name web2 -p 8081:80 nginx
```

실패해도 web2가 Created 상태로 남아서 먼저 지워요. 그다음 다른 문 번호로 켜요.

5. 자주 나는 오류

이미지를 쓰는 컨테이너가 남아 있으면 rmi가 안 돼요

```
~ $ docker rmi nginx
Error response from daemon: conflict: unable to delete nginx:latest (must be forced) - container
a669a15ddb87 is using its referenced image 165ddde8751e
```

뜻 꺼진 컨테이너라도 이 이미지로 만든 것이 남아 있어요.

고치기

```
~ $ docker ps -a
nginx로 만든 컨테이너 이름 찾기
~ $ docker rm exciting_mendeleev
~ $ docker rmi nginx
```

4파트 처음에 Ctrl+C로 멈춘 nginx 컨테이너가 남아 있었어요.

6. 실습

파이썬을 설치하지 않고 3.11과 3.12를 바로 비교해요

```
~ $ docker run python:3.11 python -c 'menu = {"latte": 4500}; print(f"라떼: {menu["latte"]}원")'
Unable to find image 'python:3.11' locally
…
  File "<string>", line 1
    menu = {"latte": 4500}; print(f"라떼: {menu["latte"]}원")
                                           ^^^^^
SyntaxError: f-string: unmatched '['   3.11은 이 문법을 몰라요

~ $ docker run python:3.12 python -c 'menu = {"latte": 4500}; print(f"라떼: {menu["latte"]}원")'
Unable to find image 'python:3.12' locally
…
라떼: 4500원   3.12는 돼요
```

```
~ $ docker images
IMAGE          ID             DISK USAGE   CONTENT SIZE   EXTRA
python:3.11    731670a425de       1.62GB          428MB   U
python:3.12    cad37dd20c7c       1.62GB          430MB   U
```

1파트의 kim과 lee 문제를 내 컴퓨터 하나에서 다시 봤어요.

이미지가 1GB가 넘어요. 다 쓰면 docker rmi로 지워요.

6. 실습

컨테이너를 지우면 안에 쌓인 기록도 같이 사라져요

```
~ $ docker logs web
…
172.17.0.1 - - [12/Nov/2026:00:33:34 +0000] "GET / H…
172.17.0.1 - - [12/Nov/2026:00:33:35 +0000] "GET / H…
172.17.0.1 - - [12/Nov/2026:00:33:35 +0000] "GET / H…
172.17.0.1 - - [12/Nov/2026:00:33:35 +0000] "GET / H…
```

지우기 전: 들어온 기록 4줄

```
~ $ docker stop web
~ $ docker rm web
~ $ docker run -d --name web -p 8080:80 nginx
~ $ docker logs web
/docker-entrypoint.sh: /docker-entrypoint.d/ is not …
…
```

새로 켠 뒤: GET 줄이 없어요

다음 챕터 30에서

켜 둔 컨테이너 안에 들어가 보고,
볼륨으로 컨테이너 밖에 데이터를 남겨요.

정리

한 장 요약

확인

`docker --version`
설치된 버전

`docker run hello-world`
설치 확인

이미지 (틀)

`docker pull 이름:태그`
Hub에서 받기

`docker images`
받은 이미지 목록

`docker rmi 이름`
이미지 지우기

컨테이너 (붕어빵)

`docker run -d --name 이름 -p 8080:80 이미지`
새로 만들어 뒤에서 켜기

`docker ps / docker ps -a`
켜진 것 / 전부

`docker stop 이름 / docker start 이름`
끄기 / 다시 켜기

`docker restart 이름`
껐다가 다시 켜기

`docker rm 이름`
지우기 (끈 다음에)

`docker logs 이름`
남긴 기록 보기

`docker run -it --name 이름 이미지 bash`
안에 들어가기 (exit로 나오기)

이미지는 Docker Official Image 마크부터 골라요. 명령은 터미널, 확인은 Docker Desktop이에요.